

1. What is Databricks Delta lake?

Delta Lake is an open-source storage framework developed by Databricks that brings reliability, consistency, and performance improvements to Data Lakes.

Traditionally, organizations stored large volumes of data in Data Lakes using cloud storage services such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage. While Data Lakes were highly scalable and cost-effective, they lacked many features found in traditional databases, such as transaction management, data consistency, and version control.

In simple words: **Delta Lake is a Data Lake with database-like capabilities.**

It combines the scalability and flexibility of a Data Lake with the reliability and performance of a database.

Why Was Delta Lake Created?

Before Delta Lake, organizations faced several challenges while working with Data Lakes.

- Data Inconsistency
- No Transaction Support
- Poor Data Quality
- No Data Versioning
- Slow Processing

What Exactly is Delta Lake?

Delta Lake is a storage layer built on top of existing Data Lakes.

It stores data in:

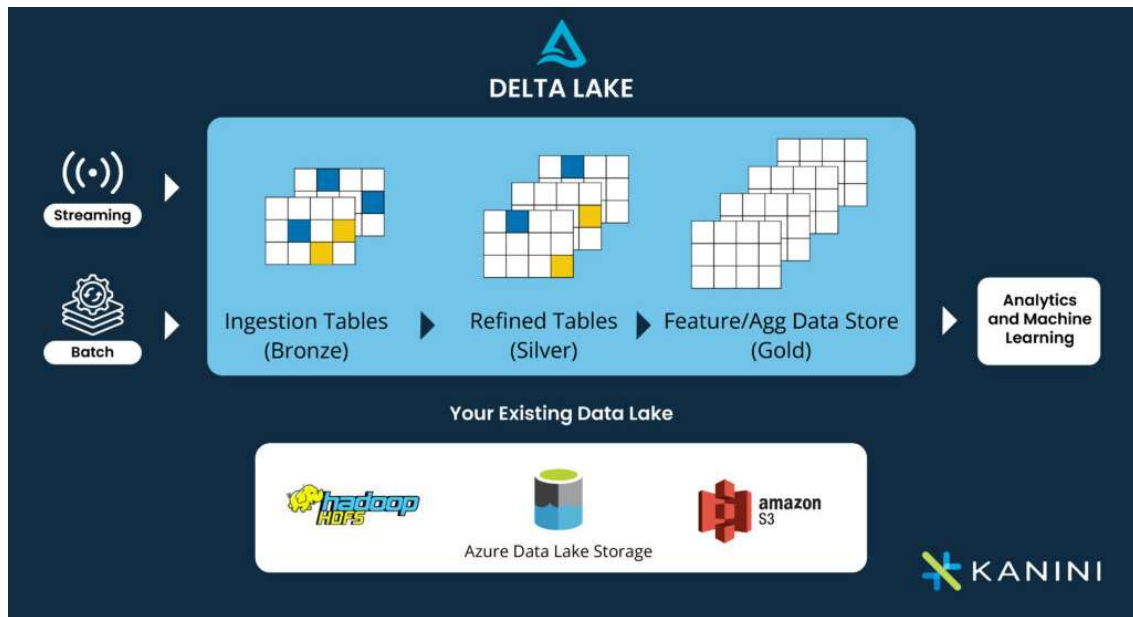
- Parquet files
- Transaction logs

The transaction log tracks every change made to the data.

This enables:

- ACID transactions
- Version control
- Data recovery
- Better performance

Delta Lake Architecture



A Delta Table consists of two major components.

1. Data Files

Actual records are stored in Parquet format.

Example:

```
sales_data/  
  part-0001.parquet  
  part-0002.parquet  
  part-0003.parquet
```

Parquet is a columnar storage format optimized for analytics.

2. Delta Log

Every Delta Table contains a special folder:

```
_delta_log/
```

Example:

```
sales_data/  
  part-0001.parquet  
  part-0002.parquet
```

_delta_log/

000000.json

000001.json

000002.json

The Delta Log records:

- Insert operations
- Update operations
- Delete operations
- Schema changes
- Transaction history

This log acts as the brain of Delta Lake.

Key Features of Delta Lake

1. ACID Transactions

One of the biggest advantages of Delta Lake is ACID compliance.

ACID stands for:

1. Atomicity

Either the entire transaction succeeds or fails.

Example:

A sales file containing 10,000 records is being loaded.

If the process fails after loading 5,000 records:

- Entire operation is rolled back.
- Partial data is not saved.

Consistency

Data always remains valid.

Invalid updates are rejected.

Isolation

Multiple users can work simultaneously without conflicts.

Example:

- User A updates sales records.
- User B reads sales reports.

Both operations occur safely.

Durability

Once data is committed, it remains permanently stored.

Even after system failures.

2. Schema Enforcement

Traditional Data Lakes allow any structure.

Example:

Existing Schema

id

name

salary

New Incoming File

id

name

salary

address

Without schema validation, data becomes inconsistent. Delta Lake checks incoming data against the table schema.

If the schema doesn't match:

Write Operation Failed

This prevents bad data from entering the system.

3. Schema Evolution

Sometimes schema changes are intentional.

Example:

A company decides to add:

address

column to customer data.

Delta Lake supports controlled schema changes through Schema Evolution.

This allows new columns to be added safely.

4. Time Travel

Time Travel is one of Delta Lake's most powerful features.

Every modification creates a new table version.

Example:

Version	Action
0	Original Data
1	New Records Added
2	Salary Updated
3	Records Deleted

if Version 3 contains errors:

You can access Version 2 instantly.

Example:

```
SELECT * FROM employees
```

```
VERSION AS OF 2;
```

Benefits:

- Data recovery
- Auditing
- Debugging
- Historical analysis

5. Data Versioning

Delta Lake automatically maintains versions.

Benefits include:

Audit Trail

Track who changed what and when.

Rollback Capability

Restore previous versions.

Reproducibility

Machine Learning experiments can use the exact same data version repeatedly.

6. Batch and Streaming Support

Most systems support either:

Batch Processing

Example:

Daily sales reports.

or

Stream Processing

Example:

Real-time transactions.

Delta Lake supports both on the same table.

Example:

- Historical sales data
- Real-time sales data

can coexist in a single Delta Table.

7. Scalable Metadata Handling

Traditional Data Lakes struggle when millions of files exist.

Delta Lake uses transaction logs and optimized metadata management.

Benefits:

- Faster file discovery
- Faster queries

- Better scalability

8. Data Reliability

Delta Lake guarantees:

- No corrupted tables
- No partial writes
- No inconsistent reads

This makes it suitable for enterprise applications.

How Delta Lake Works Internally

When data is inserted:

```
df.write.format("delta").save("/delta/sales")
```

Delta Lake performs:

Step 1: Writes data into Parquet files.

Step 2: Updates transaction log.

Step 3: Creates a new table version.

Step 4: Commits transaction.

Only after successful completion is data made visible.

This ensures consistency.

Delta Lake in Databricks

Databricks uses Delta Lake as its default storage format.

Typical workflow:

Step 1: Load Data

```
df = spark.read.csv("sales.csv")
```

Step 2: Transform Data

```
clean_df = df.filter(df.revenue > 0)
```

Step 3: Save as Delta Table

```
clean_df.write \
```

```
.format("delta") \  
.mode("overwrite") \  
.save("/delta/sales")
```

Step 4: Query Using SQL

```
SELECT * FROM sales;
```

Advantages of Delta Lake

- Improved Data Quality
- Faster Processing
- Reliable Transactions
- Time Travel
- Cost Effective
- Scalable
- Unified Analytics

Real-World Example

Consider an online shopping company.

Every day it generates:

- Customer data
- Orders
- Product information
- Website logs
- Payment transactions

All data is stored in a Data Lake.

To ensure:

- Reliable updates
- Historical tracking
- High-performance analytics

the company stores critical datasets in Delta Lake.

Analysts can:

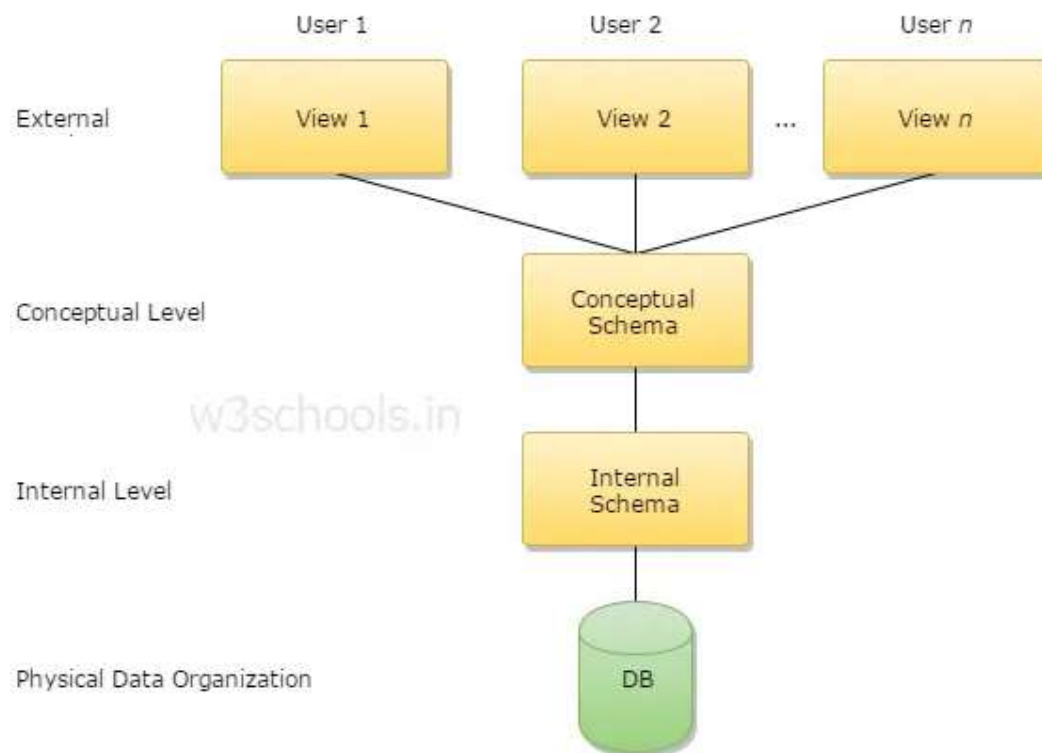
- Query data quickly
- Recover deleted records

- Track historical changes
- Build machine learning models

2. Database vs Data Warehouse vs Data Lake vs Delta Lake

To understand modern Data Engineering, it is important to understand the evolution from Database → Data Warehouse → Data Lake → Delta Lake. Each technology was developed to solve the limitations of the previous one. As organizations generated larger volumes of data and required more advanced analytics, the traditional database systems were no longer sufficient. This led to the development of data warehouses, data lakes, and eventually Delta Lake.

1. Database



A database is a structured system used to store, organize, manage, and retrieve data efficiently. Databases are primarily designed for Online Transaction Processing (OLTP) workloads, where thousands or millions of small transactions occur every day. Examples include banking transactions, employee attendance systems, online shopping orders, and hospital management systems.

In a database, data is stored in tables consisting of rows and columns. Each table has a predefined schema, meaning the structure of the data must be defined before

data can be inserted. Because of this strict structure, databases maintain high levels of consistency and reliability.

For example, in an employee management system, information such as Employee ID, Name, Department, and Salary is stored in a table. Whenever a new employee joins, a new row is inserted into the table.

Characteristics of a Database

They support ACID properties, which guarantee reliable transactions:

- Atomicity ensures that a transaction either completes fully or does not happen at all.
- Consistency ensures that data remains valid after every transaction.
- Isolation ensures that multiple transactions do not interfere with one another.
- Durability ensures that committed data remains safe even after system failures.

Databases are optimized for frequent insertions, updates, deletions, and lookups.

Advantages of Databases

Databases provide fast transaction processing and strong data consistency. They offer robust security mechanisms, backup facilities, indexing, and user access controls. They are ideal for applications that require real-time updates and accurate record management.

For example:

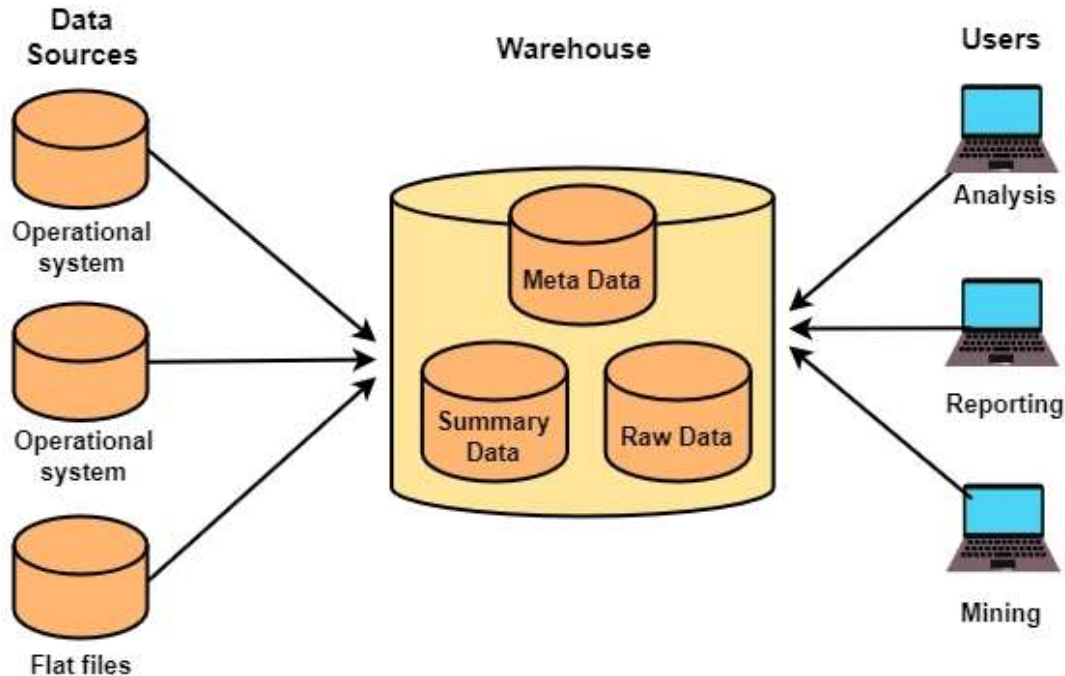
- Banking Systems
- Employee Management Systems
- Student Information Systems
- E-commerce Applications

Limitations of Databases

As data volume grows into terabytes or petabytes, traditional databases become expensive and difficult to scale. They are primarily designed for structured data and struggle with unstructured formats such as videos, images, audio files, and social media content.

This limitation led organizations to create specialized systems for analytics and reporting, known as Data Warehouses.

2. Data Warehouse



A Data Warehouse is a centralized repository designed specifically for business intelligence, reporting, and analytical processing. Unlike databases, which focus on daily transactions, data warehouses focus on analyzing historical data to support decision-making.

Organizations often collect data from multiple operational databases and combine it into a single warehouse. This allows business analysts and management teams to perform complex queries without affecting the performance of operational systems.

For example, a retail company may have separate databases for sales, inventory, customers, and employees. A data warehouse combines all this information into one place for analysis.

Purpose of a Data Warehouse

The primary goal of a data warehouse is to provide a single source of truth for reporting and analytics.

A typical flow looks like this:

Operational Databases



ETL



Data Warehouse



Reports & Dashboards

Data is extracted from operational systems, transformed into a standardized format, and loaded into the warehouse through an ETL process.

Characteristics of a Data Warehouse

A data warehouse stores historical data collected over months or years. It is optimized for complex analytical queries that involve aggregations, calculations, and trend analysis.

For example, a company might analyze:

- Revenue growth over the last five years
- Top-selling products
- Customer purchasing patterns
- Employee productivity trends

Unlike operational databases, data warehouses are designed for heavy read operations rather than frequent updates.

Advantages of Data Warehouses

Data warehouses provide extremely fast analytical performance because data is organized specifically for reporting. They help businesses generate dashboards, KPI reports, and strategic insights.

Benefits include:

- Historical analysis
- Business intelligence

- Data consistency across departments
- Centralized reporting

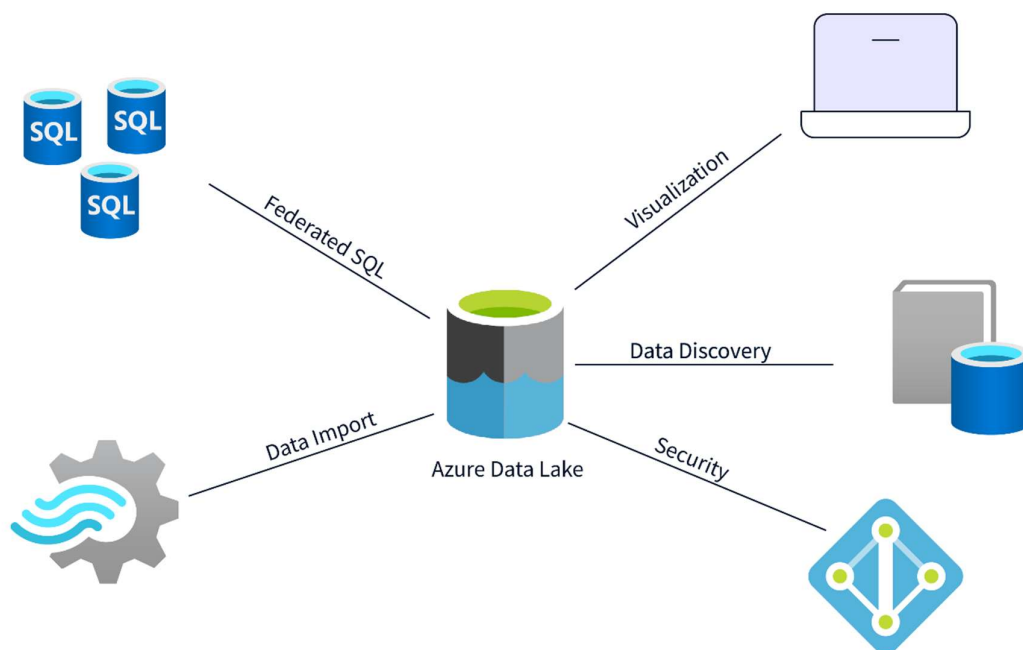
Limitations of Data Warehouses

Although powerful, data warehouses can be expensive to build and maintain. Data must be cleaned and transformed before loading, which increases complexity.

Additionally, traditional warehouses mainly handle structured data and are not ideal for storing large volumes of images, videos, logs, and sensor data.

As organizations began generating massive amounts of diverse data, Data Lakes emerged as a solution.

3. Data Lake



A Data Lake is a centralized storage repository that allows organizations to store vast amounts of data in its original raw format.

Unlike databases and warehouses, data lakes do not require a predefined schema. Data can be stored exactly as it arrives.

This includes:

- CSV files

- JSON documents
- XML files
- Images
- Videos
- Log files
- IoT sensor data
- Social media feeds

The idea behind a data lake is simple:

Store everything first and decide later how to use it.

Schema-on-Read Concept

Traditional databases use a Schema-on-Write approach, where the structure must be defined before data is stored.

Data lakes use Schema-on-Read, meaning the schema is applied only when data is read for analysis.

This flexibility allows organizations to collect data without worrying about its final use case.

Characteristics of a Data Lake

Data lakes are built on low-cost cloud storage systems and can scale almost infinitely.

They support:

- Structured Data
- Semi-Structured Data
- Unstructured Data

This makes them highly suitable for big data and machine learning projects.

Advantages of Data Lakes

Data lakes provide tremendous flexibility because they can store any type of data. Storage costs are significantly lower compared to traditional databases and warehouses.

Benefits include:

- Unlimited scalability
- Low-cost storage
- Support for all data formats
- Ideal for AI and Machine Learning

Problems with Traditional Data Lakes

No ACID Transactions

Multiple users writing simultaneously can lead to corrupted data.

Poor Data Quality

Without schema enforcement, inconsistent and invalid data may enter the system.

No Version Control

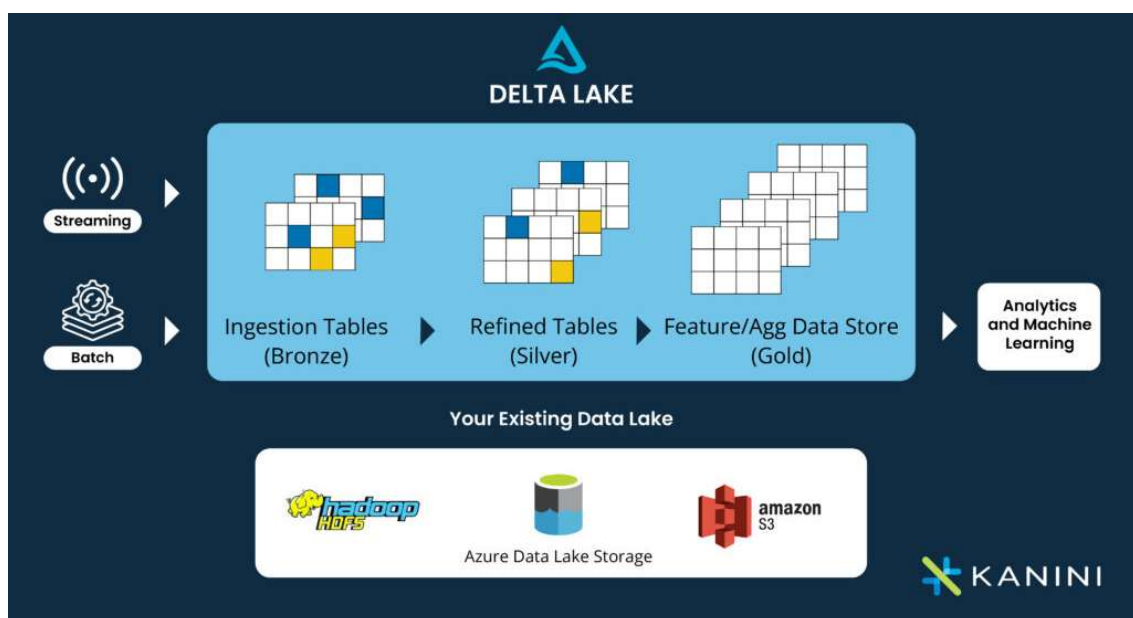
Accidental deletions or modifications are difficult to recover.

Metadata Challenges

Managing millions of files becomes increasingly difficult.

Over time, poorly managed data lakes often become what is known as a Data Swamp, where finding reliable data becomes nearly impossible.

4. Delta Lake



Delta Lake is an open-source storage layer developed by Databricks that adds database-like reliability and performance features to a Data Lake. Delta Lake is built on top of cloud storage systems and uses Parquet files for storing data along with a transaction log called `_delta_log`.

Why Delta Lake Was Created

Organizations loved the scalability and low cost of Data Lakes but struggled with reliability issues.

Delta Lake was created to combine the strengths of both databases and data lakes.

Delta Lake Architecture

A Delta Table contains two major components:

Parquet Files

These files store the actual data records.

Delta Transaction Log

A special directory called `_delta_log` stores metadata about every operation performed on the table.

The log records:

- Insert operations
- Update operations
- Delete operations
- Schema changes
- Table versions

ACID Transactions

One of the most important features of Delta Lake is ACID transaction support.

Suppose a process is loading one million sales records.

If the process fails after loading 500,000 records:

- Traditional Data Lake → Partial data remains.
- Delta Lake → Entire transaction rolls back automatically.

Schema Enforcement

Delta Lake validates incoming data against the table schema.

If an incoming dataset contains unexpected columns or incompatible data types, Delta Lake rejects the write operation.

This prevents poor-quality data from entering the system.

Schema Evolution

In real-world applications, schemas often change.

Delta Lake supports controlled schema evolution, allowing new columns to be introduced safely.

Time Travel

Time Travel is one of Delta Lake's most popular features.

Every change creates a new version of the table.

This allows users to query previous versions and restore data if needed.

For example, if someone accidentally deletes records, a previous version can be retrieved instantly.

This feature is extremely valuable for auditing, debugging, and compliance requirements.

Batch and Streaming Support

Most traditional systems support either batch processing or stream processing.

Delta Lake supports both simultaneously.

For example:

- Historical sales data can be stored in a Delta Table.
- Real-time sales transactions can be continuously appended to the same table.

Advantages of Delta Lake

Delta Lake combines the best features of databases, warehouses, and lakes.

Key benefits include:

- ACID Transactions
- Schema Enforcement

- Schema Evolution
- Time Travel
- Data Versioning
- High Scalability
- Streaming Support
- Improved Performance
- Lower Storage Costs

Final Comparison

Aspect	Database	Data Warehouse	Data Lake	Delta Lake
Primary Purpose	Transactions	Analytics	Raw Data Storage	Reliable Analytics
Data Type	Structured	Structured	All Types	All Types
ACID Support	Yes	Yes	No	Yes
Schema Enforcement	Yes	Yes	No	Yes
Historical Analysis	Limited	Excellent	Moderate	Excellent
Time Travel	No	Limited	No	Yes
Storage Cost	High	High	Low	Low
Big Data Support	Low	Medium	High	High
Streaming Support	Limited	Limited	Yes	Yes
Scalability	Medium	High	Very High	Very High